39. Арифметические операторы

На этом уроке мы рассмотрим арифметические операторы и их использование в языке C++.

Оглавление:

1. Унарные арифметические операторы
2. Бинарные арифметические операторы
3. Деление целых чисел и чисел типа с плавающей точкой
4. Использование оператора static_cast в операциях деления
5. Деление с остатком
6. Отрицательные числа в операциях деления до C++11
7. Арифметические операторы присваивания
8. Где оператор возведения в степень?
9. Тест

Унарные арифметические операторы

Унарные операторы — это операторы, которые применяются только к одному операнду. Существуют два унарных арифметических оператора: плюс (`+`) и минус (`-`).

Оператор	Символ	Пример	Операция
Унарный плюс	`+`	`+x`	Значение x
Унарный минус	`-`	`-x`	Отрицательное значение x

Унарный оператор `+` возвращает значение операнда. Другими словами, `+5 = 5` или `+x = x`. Унарный плюс вам, скорее всего, не придется использовать. Его по большей части добавили в качестве симметрии с унарным оператором минус. Унарный оператор минус возвращает операнд, умноженный на `-1`. Например, если `x = 5`, то `-x = -5`.

Оба этих оператора пишутся непосредственно перед самим операндом, без пробела (`-x`, а не `- x`).

Не следует путать унарный оператор минус с бинарным оператором вычитания, хоть они и используют один и тот же символ. Например, в выражении `x = 5 - -3;`, первый минус — это оператор вычитания, а второй — унарный минус.

Бинарные арифметические операторы

Бинарные операторы — это операторы, которые применяются к двум операндам (слева и справа). Существует 5 бинарных операторов.

Оператор	Символ	Пример	Операция
Сложение	`+`	`x + y`	x плюс y
Вычитание	`-`	`x - y`	x минус y
Умножение	`*`	`x * y`	x умножить на y
Деление	`/`	`x / y`	x разделить на y
Деление с остатком	`%`	`x % y`	Остаток от деления x на y

Операторы сложения, вычитания и умножения работают так же, как и в обычной математике. А вот деление и деление с остатком рассмотрим детально.

Деление целых чисел и чисел типа с плавающей точкой

Оператор деления имеет два режима. Если оба операнда являются целыми числами, то оператор выполняет целочисленное деление. Т.е. любая дробь (большая/меньшая) отбрасывается и возвращается целое значение без остатка, например, `7 / 4 = 1`.

Если один или оба операнда **типа с плавающей точкой**, то тогда будет выполняться деление типа с плавающей точкой. Здесь уже дробь присутствует. Например, выражения `7.0 / 3 = 2.333`, `7 / 3.0 = 2.333` или `7.0 / 3.0 = 2.333` имеют один и тот же результат.

Попытки деления на `0` (или на `0.0`) станут причиной сбоя в вашей программе, и это правило не следует забывать!

Использование оператора static_cast в операциях деления

На **уроке о символьном типе данных char** мы уже использовали оператор static_cast для вывода ASCII-символов в виде целых чисел.

Аналогичным образом мы можем использовать static_cast для конвертации целого числа в число типа с плавающей точкой. Таким образом, вместо целочисленного деления выполнится деление типа с плавающей точкой. Например:

```
#include <iostream>

int main()
{
    int x = 7;
    int y = 4;

    std::cout << "int / int = " << x / y << '\n';
    std::cout << "double / int = " << static_cast<double>(x) / y << '\n';
    std::cout << "int / double = " << x / static_cast<double>(y) << '\n';
    std::cout << "double / double = " << static_cast<double>(x) / static_cast<double>(y) << '\n';

    return 0;
}
```

Результат выполнения программы:

```
int / int = 1
double / int = 1.75
int / double = 1.75
double / double = 1.75
```

Деление с остатком

Оператор деления с остатком (`%`) работает только с целочисленными операндами и возвращает остаток от целочисленного деления. Например:

→ *Пример №1:* `7 / 4 = 1` с остатком `3`, таким образом, `7 % 4 = 3`.

→ *Пример №2:* `25 / 7 = 3` с остатком `4`, таким образом, `25 % 7 = 4`. Остаток составляет не дробь, а целое число.

→ *Пример №3:* `36 % 5 = 7` с остатком `1`. В числе 36 только 35 делится на 5 без остатка, поэтому `36 - 35 = 1`, `1` — это остаток и результат.

Данный оператор чаще всего используют для проверки деления без остатка одних чисел на другие. Если `x % y == 0`, то `x` делится на `y` без остатка.

Например, мы хотим написать программу, которая выводит числа от 1 до 100 по 20 значений в каждой строке. Мы можем использовать оператор деления с остатком для создания разрыва строк. Несмотря на то, что мы еще не рассматривали цикл while, в следующей программе всё максимально просто и понятно:

```
#include <iostream>

int main()
{
    // Переменная count хранит текущее число для вывода
    int count = 1; // начинаем с 1

    // Пока текущее значение count не 100 включительно, пока count не будет равно 100
    while (count <= 100)
    {
        std::cout << count << " "; // выводим текущее число

        // Если count делится на 20 без остатка, то вставляем разрыв строки и продолжаем с новой строки
        if (count % 20 == 0)
            std::cout << "\n";

        count = count + 1; // переходим к следующему числу
    } // конец while

    return 0;
} // конец main()
```

Результат выполнения программы:

```
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20
21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40
41 42 43 44 45 46 47 48 49 50 51 52 53 54 55 56 57 58 59 60
61 62 63 64 65 66 67 68 69 70 71 72 73 74 75 76 77 78 79 80
81 82 83 84 85 86 87 88 89 90 91 92 93 94 95 96 97 98 99 100
```

О while мы еще поговорим на соответствующем уроке.

Отрицательные числа в операциях деления до C++11

До **C++11**, если любой из операндов целочисленного деления является отрицательным, то компилятор округляет результат самостоятельно! Например, результатом `-5 / 2` может быть либо `-3`, либо `-2`. Однако большинство современных компиляторов округляют числа в сторону нуля (например, в `-5 / 2` результатом будет `-2`). В спецификации C++11 определили, что компилятор должен всегда округлять к нулю (или, проще говоря, просто отбрасывать дробь).

Также до C++11, если один из операндов оператора деления с остатком является отрицательным, то результат может быть как положительным, так и отрицательным! Например, результатом `-5 % 2` может быть как `1`, так и `-1`. В спецификации C++11 решили сделать так, чтобы результат `a % b` был того же знака, что и значение `a`.

Арифметические операторы присваивания

Оператор	Символ	Пример	Операция
Присваивание	`=`	`x = y`	Присваиваем значение y переменной x
Сложение с присваиванием	`+=`	`x += y`	Добавляем y к x
Вычитание с присваиванием	`-=`	`x -= y`	Вычитаем y из x
Умножение с присваиванием	`*=`	`x *= y`	Умножаем x на y
Деление с присваиванием	`/=`	`x /= y`	Делим x на y
Деление с остатком и с присваиванием	`%=`	`x %= y`	Присваиваем остаток от деления x на y переменной x

До этого момента, когда нам нужно было добавить число 5 к определенной переменной, мы делали следующее:

```
x = x + 5;
```

Это работает, но требуется два оператора для выполнения.

Так как стейтменты типа `x = x + 5` являются очень распространенными, то C++ предоставляет 5 арифметических операторов присваивания для нашего удобства. Вместо `x = x + 5`, мы можем записать:

```
x += 5;
```

Вместо:

```
x = x * y;
```

Мы можем записать:

```
x *= y;
```

Где оператор возведения в степень?

В языке C++ вместо оператора возведения в степень есть **функция pow()**, которая находится в **заголовочном файле** cmath. pow(base, exponent) эквивалентно $base^{exponent}$. Стоит отметить, что параметры pow() имеют тип double, поэтому вы можете использовать не только целые числа, но и дробные. Например:

```
#include <iostream>
#include <cmath> // подключаем pow()

int main()
{
    std::cout << "Enter the base: ";
    double base;
    std::cin >> base;

    std::cout << "Enter the exponent: ";
    double exp;
    std::cin >> exp;

    std::cout << base << "^" << exp << " = " << pow(base, exp) << "\n";

    return 0;
}
```

Тест

Задание №1

Вычислите результат следующего выражения: `6 + 5 * 4 % 3`.

Задание №2

Напишите программу, которая просит пользователя ввести целое число, а затем сообщает, является ли его число чётным или нечётным. Напишите функцию isEven(), которая возвращает `true`, если целое число является чётным. Используйте оператор деления с остатком, чтобы определить чётность числа.

> ***Подсказка:*** Используйте ветвление if и оператор сравнения (`==`).

Ответ №1

Поскольку операторы `*` и `%` имеют более высокий **приоритет**, чем оператор `+`, то оператор `+` будет выполняться последним. Мы можем переписать наше выражение следующим образом: `6 + (5 * 4 % 3)`. Операторы `*` и `%` имеют одинаковый приоритет, но их ассоциативность слева направо, так что левый оператор будет выполняться первым. Получается `6 + ((5 * 4) % 3)`.

`6 + ((5 * 4) % 3) = 6 + (20 % 3) = 6 + 2 = 8`

Ответ: 8.

Ответ №2

```
#include <iostream>

bool isEven(int x)
{
    // Если x % 2 == 0, то x - это четное число
    return (x % 2) == 0;
}

int main()
{
    std::cout << "Enter an integer: ";
    int x;
    std::cin >> x;

    if (isEven(x))
        std::cout << x << " is even\n";
    else
        std::cout << x << " is odd\n";

    return 0;
}
```